

Manejando excepciones

Recogiendo una excepción usando try y catch

El término excepción significa "**condición excepcional**" y es un suceso que altera el flujo normal de un programa. Muchas cosas pueden provocar una excepción en Java: fallos de hardware, errores de codificación, saturación de recursos, etc...

Cuando un evento excepcional se produce en Java, se dice que una excepción es lanzada (**thrown**). El código responsable de hacer algo con la excepción es denominado "manejador de la excepción" (**exception handler**), y este recoge (**catches**) la excepción lanzada.

El manejador de excepciones trabaja transfiriendo la ejecución del programa a un manejador de excepción apropiado cuando se produce una excepción. Por ejemplo, si invocamos a un método que abre un fichero pero el fichero no puede ser abierto, la ejecución del método parará, y el código que escribimos para tratar con esta situación será ejecutado. En consecuencia, necesitamos una forma de decir a la máquina virtual de Java que código se ejecutara cuando ocurra una cierta excepción. Para hacer esto, usamos las palabras **try** y **catch**. El try se usa para definir un bloque de código en el cuál las excepciones pueden ocurrir. Uno o más catch hacen corresponder una excepción específica con un bloque de código que la maneja.

```
try{
    // Primera línea de la "región salvaguardada", gobernada por la
    // palabra clave try.
    // Aquí se pondrá el código que puede producir alguna clase de
    // excepción. Podemos poner una o varias líneas de código.
}
catch(Mi primera excepción){
    // Aquí código que maneja la excepción.
}
catch(Mi segunda excepción){
    // Aquí código que maneja la excepción.
}
// Fuera es resto del código que no tiene peligro potencial.
```

Los bloques **catch** siguen inmediatamente al bloque **try**. Si tenemos uno o más bloques **catch**, estos deben ir inmediatamente después del bloque **try**.

Uno de los beneficios de usar el manejo de excepciones es que es código que maneja una excepción particular que puede ocurrir en la región gobernada sólo necesita ser escrito una vez.

Usando finally

Un bloque **finally** contiene código que será ejecutado **siempre** en algún punto después del bloque **try**, independientemente de si se produce una excepción o no. Incluso si hay una sentencia **return** dentro del bloque **try**, el bloque **finally** se ejecutará justo después de encontrar al **return**, y antes de que el **return** se ejecute.

El bloque **finally** es el sitio indicado para cerrar ficheros, cerrar conexiones de red, y en general, ejecutar cualquier código de limpieza a fin de no tener que hacerlo en cada uno de los bloques **catch**.

```
try{
    // Primera línea de la "región salvaguardada"
}
catch(Mi primera excepción){
    // Aquí código que maneja la excepción.
}
catch(Mi segunda excepción){
    // Aquí código que maneja la excepción.
}
finally{
    // Aquí código para liberar cualquier recurso comprometido
    // en el bloque try.
}
// Más código aquí.
```

- No es obligatorio escribir el bloque **finally**.
- Es ilegal usar la cláusula **try** sin tener o una cláusula **catch** o una **finally**.
- Una cláusula **try** sola dará lugar a un error de compilación.
- Cualquier cláusula **catch** debe de seguir inmediatamente al bloque **try**.
- Cualquier cláusula **finally** debe seguir inmediatamente a el último bloque **catch** (o debe seguir inmediatamente al bloque **try** en caso de no haber bloque **catch**).
- Es legal omitir la cláusula **catch** o la **finally**, pero no ambas a la vez.

Propagando excepciones no recogidas

Imaginemos el caso de una función principal que invoca a un método A, que a su vez invoca a un método B, y este a su vez invoca a un método C.

Si en el método C se lanza una excepción **y este método no la recoge** (no la trata con **try-catch**), **la excepción será propagada** al método B, que puede tratar la excepción o en caso contrario pasarla al método A. Este a su vez puede tratarla o pasársela a la

función principal (último eslabón en donde se originó la llamada). Si la función principal no trata la excepción, la excepción nunca será recogida y esto provocará la parada de la ejecución de la aplicación.

```
class TextEx{
    static void doMoreStuff( ){
        int x = 5/0; // No se puede dividir entre 0, esto provoca que se lance una
    }           // ArithmeticException!!!
    static void doStuff( ){
        doMoreStuff( );
    }
    public static void main(String[ ] args){
        doStuff( );
    }
}
```

Definiendo excepciones

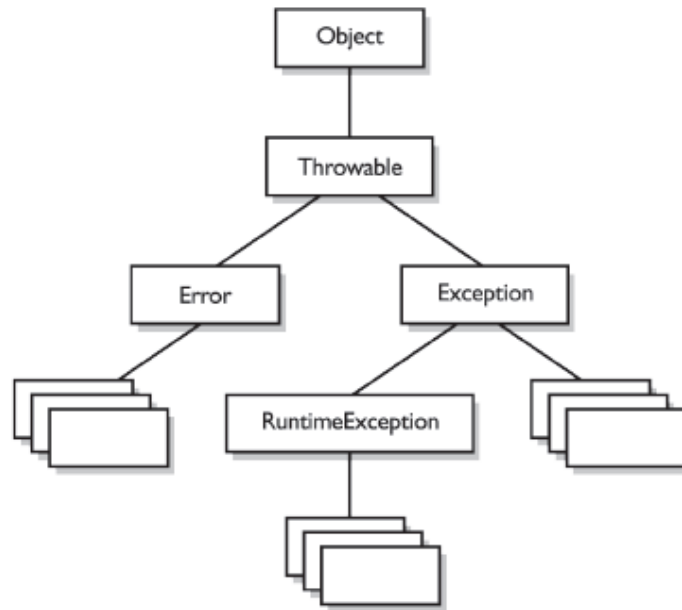
Cada exception es una instancia de una clase que tiene a la **clase Exception** en su jerarquía de herencia (como superclase). En otras palabras, las exceptions son siempre subclases de **java.lang.Exception**.

Cuando una excepción es lanzada, se crea una instancia de un subtipo particular de Exception, que es pasada al manejador de la excepción como un argumento para la cláusula catch.

```
try{
    // Código aquí.
}
catch (ArrayIndexOutOfBoundsException e){
    e.printStackTrace( );
}
```

La jerarquía de la clase Exception

Todas las clases que crean excepciones son subclases de la clase **Exception**. Esta clase descende de la clase **Throwable**, que a su vez descende de la clase **Object**.



Hay dos clases que derivan de **Throwable**: **Exception** y **Error**.

Las clases que derivan de **Error** representan situaciones inusuales no causadas por errores del programa e indican cosas que no deberían suceder durante la ejecución del programa, por ejemplo, que la JVM se quede sin memoria. **Generalmente la aplicación no se podrá recuperar de un Error.** Los **Error** técnicamente no son Exceptions, ya que no derivan de la clase Exception.

En general, una excepción representa algo que ocurre no como el resultado de un error de programación, sino porque algún recurso no está disponible o alguna otra condición requerida para la ejecución no está presente.

Por ejemplo, si nuestra aplicación está hecha para comunicarse con otra aplicación en otro ordenador que no está respondiendo, esta es una excepción que no está causada por un error en el código.

La clase **Throwable** proporciona a sus descendientes algunos métodos útiles para los manejadores de excepciones. Uno de ellos es **printStackTrace()**.

Como veremos más adelante, los tipos **Exception**, **Error**, **RuntimeException** y **Throwable** pueden ser lanzados usando la palabra reservada **throw**, y todos pueden ser manejados (aunque raramente se maneja otra cosa que los subtipos de Exception).

Manejando una jerarquía de clases de excepciones completa

La palabra clave **catch** permite especificar un tipo particular de excepción para ser recogida. De hecho, se puede recoger más de un tipo de excepción en una única cláusula **catch**. Si la subclase de **Exception** que especificamos en la cláusula **catch** no tiene subclases, entonces sólo se recogerán las excepciones de la clase especificada.

Sin embargo si la clase especificada tiene subclases, cualquier objeto excepción de las subclases de la clase especificada también será recogido.

```
try{
    // Código que puede lanzar cualquier tipo de excepción.
}
catch (IndexOutOfBoundsException e){ // Únicamente trata este tipo de excepción.
    e.printStackTrace( );
}
```

```
try{
    // Código que puede lanzar cualquier tipo de excepción.
}
catch (Exception e){ // Trata cualquier tipo de excepción.
    e.printStackTrace( );
}
```

Recogida de excepciones

Cuando una excepción es lanzada, Java intentará encontrar (recorriendo las cláusulas catch disponibles de arriba hacia abajo) una cláusula catch para el tipo específico de excepción. Si no encuentra ninguno, buscará un manejador para un supertipo del tipo de excepción.

Si no encuentra una cláusula catch que maneje un supertipo de la excepción, entonces esta será propagada al siguiente método (que invocó al actual).

```
import java.io.*;
public class ReadData {
    public static void main(String args[ ]) {
        try {
            RandomAccessFile raf = new RandomAccessFile("myfile.txt", "r");
            byte b[ ] = new byte[1000];
            raf.readFully(b, 0, 1000);
        }
        catch(FileNotFoundException e) {
            System.err.println("File not found");
            System.err.println(e.getMessage());
            e.printStackTrace();
        }
        catch(IOException e) { // FileNotFoundException es subclase de IOException
            System.err.println("IO Error");
            System.err.println(e.toString());
            e.printStackTrace();
        }
    }
}
```

En el ejemplo anterior hay que darse cuenta de que se escribe primero la excepción más específica y después las de propósito más general.

LOS MANIPULADORES DE LAS EXCEPCIONES MÁS ESPECÍFICAS DEBEN SIEMPRE DE ESCRIBIRSE ANTES DE LOS MANIPULADORES DE EXCEPCIONES MÁS GENERALES, SINO EL CÓDIGO NO COMPILARÁ.

Si una clase de excepción no es un subtipo o un supertipo de la otra, entonces el orden en el las cláusulas catch están colocadas no tiene importancia.

Declaración de excepciones y la interfaz pública

¿Cómo sabemos que un método puede lanzar una excepción que tenemos que recoger? Así como un método debe especificar el tipo y número de argumentos que toma y el tipo de retorno, **las excepciones que un método puede lanzar deben ser declaradas (a menos que las excepciones sean subclase de RuntimeException)**. La lista de excepciones lanzadas es parte de la interfaz pública del método. La palabra clave **throws** se utiliza para listar las excepciones que un método puede lanzar:

```
void miMetodo() throws MiException1, MiException2{  
    // Aquí el código del método.  
}
```

Este método tiene un tipo de retorno void, no acepta argumentos, y declara que puede lanzar una de dos tipos de excepciones.

Esta es la jerarquía de **RuntimeException**:

- class java.lang.RuntimeException
 - class java.lang.ArithmeticException
 - class java.lang.ArrayStoreException
 - class java.lang.ClassCastException
 - class java.lang.IllegalArgumentException
 - class java.lang.IllegalThreadStateException
 - class java.lang.NumberFormatException
 - class java.lang.IllegalMonitorStateException
 - class java.lang.IllegalStateException
 - class java.lang.IndexOutOfBoundsException
 - class java.lang.ArrayIndexOutOfBoundsException
 - class java.lang.StringIndexOutOfBoundsException
 - class java.lang.NegativeArraySizeException
 - class java.lang.NullPointerException
 - class java.lang.SecurityException
 - class java.lang.UnsupportedOperationException

Supongamos que un método A no lanza directamente una excepción, pero invoca a otro método B que sí lo hace. En este caso podemos elegir no manejar la excepción en el método A y en vez de esto sólo declararla, como si fuese el método A el que podría lanzar la excepción. Si declaramos la excepción que el método puede recibir de otro método (en este caso declaramos en el método A la excepción que puede recibir del método B), y no proporcionamos un try/catch para manejarla, entonces el método propagará la excepción de vuelta al método que invocó a nuestro método (en nuestro caso se pasaría de vuelta al método B).

En otras palabras, cualquier método que pueda lanzar una excepción debe declarar dicha excepción (a menos que la excepción sea de una subclase de RuntimeException). Esto incluye métodos que no pueden de hecho lanzar la excepción directamente, pero la pueden recibir de otro método y a su vez pasarla.

Las subclases de **RuntimeException** estén exentas de que el compilador las compruebe para ver si el programador las ha declarado (excepciones no comprobadas. Sin embargo, el resto de excepciones (no subclases de RuntimeException), son consideradas excepciones "comprobadas", por que el compilador comprobará que las hemos declarado y manejado. Una excepción comprobada debe ser recogida en alguna parte del código. Si invocamos un método que lanza una excepción comprobada pero no recogemos la excepción comprobada en alguna parte, el código no compilará. Por eso se llaman excepciones comprobadas: el compilador las comprueba para asegurar que son manejadas o declaradas.

Cada método debe o bien manejar todas las excepciones comprobadas mediante la cláusula catch adecuada para cada excepción comprobada o bien listar cada excepción comprobada no manejada como una excepción lanzada (**Regla manejar o declarar**).

```
import java.io.IOException;
public class Test {
    static void doStuff() {
        try {
            doMore();
        } catch (IOException ex) {
            ex.getMessage();
        } // Podemos optar por declarar o manejar la excepción. En este caso la
           // controlamos con try/catch y ya no se propagará más allá.
    }

    static void doMore() throws IOException {           //Se declara la excepción.
        throw new IOException(); // lanza una excepción de la clase IOException.
    }
    public static void main (String[] args)
    {
        doStuff();
    }
}
```

Lo siguiente compila, pero ¿Qué ocurrirá al ejecutarse?

```
import java.io.IOException;
public class Test {
    static void doStuff() throws IOException {
        doMore();
    }
    static void doMore() throws IOException {
        throw new IOException();
    }
    public static void main (String[] args) throws IOException
    {
        doStuff();
    }
} //Todos los métodos declaran la excepción pero ninguno la maneja.
```

Comparar con:

```
import java.io.IOException;
public class Test {
    static void doStuff() throws IOException {
        doMore();
    }
    static void doMore() throws IOException {
        throw new IOException();
    }
    public static void main (String[] args)
    {
        try
        {
            doStuff();
        }
        catch(IOException e){ }
    }
}
```

Las excepciones Runtime son conocidas como **excepciones no comprobadas**. Todas las demás son excepciones comprobadas, y no derivan de **java.lang.RuntimeException**. Una excepción comprobada debe ser recogida en alguna parte del código. Si invocamos a un método que lanza una excepción comprobada pero no recogemos esa excepción comprobada en algún sitio, en nuestro código, el código no compilará.

En cuanto a los errores, los objetos de la clase **Error** no son objetos del tipo Exception, aunque ambos representan condiciones excepcionales. Ambos tienen la superclase **Throwable**, por lo que ambos pueden ser lanzados usando la palabra reservada **throw**.

Cuando se lanza un Error o una subclase de Error, **no son comprobados**, por lo que no se requiere recoger (catch) los objetos de la clase error o de sus subtipos.

```
class TestEx {
    static void badMethod() { // No es necesario declarar ningún Error
        doStuff()
    }
    static void doStuff() { // No es necesario declarar ningún Error
        try {
            throw new Error();
        }
        catch(Error me) {
            throw me; // Lo recoge, pero lo relanza.
        }
    }
    public static void main (String [] args) {
        badMethod();
    }
}
```

Creando una excepción propia

Podemos crear nuestras propias excepciones, basta con extender la clase **Exception** o a alguna de sus subclases.

```
class IndiceFueraDeRangoException extends Exception{
    public IndiceFueraDeRangoException( ){
        super( );
    }
    public IndiceFueraDeRangoException(String s){
        super(s);
    }
    public String toString( ){
        return ( "Se ha producido la excepción " + this.getClass( ).getName( ) + "\n" +
            " Con el siguiente mensaje: " + this.getMessage( ) + "\n");
    }
}
```

La clase anterior define una excepción propia, ¿Cómo deberíamos usarla?

```

public double accesoInd(double[ ] v, int indice) throws IndiceFueraDeRangoException
{
    try{
        if ( (0<= indice) && (indice<v.length)){
            return v[indice];
        }
        else{
            //Caso excepcional:
            throw new IndiceFueraDeRangoException("El índice " + indice + "no es una
            posición válida.");
        }
    }
    catch(IndiceFueraDeRangoException mi_excepcion){
        System.out.println(mi_excepcion.toString( ));
        System.out.println(mi_excepcion.getMessage( ));
    }
}

```

Excepciones y errores corrientes

Podemos definir dos amplias categorías de excepciones y errores:

- **Excepciones y errores JVM:** Aquellas excepciones o errores que son exclusiva o con más lógica lanzados por la Máquina Virtual de Java, por ejemplo `NullPointerException` o `StackOverflowError`.
- **Excepciones y errores programáticos:** Aquellas excepciones que son lanzadas explícitamente por una aplicación y/o los programadores que usan la API. Por ejemplo `NumberFormatException`, `IllegalFormatException` o `AssertionError`.

A continuación una tabla con las descripciones y las fuentes de las excepciones más frecuentes:

Exception	Description	Typically Thrown
<code>ArrayIndexOutOfBoundsException</code> (this chapter)	Thrown when attempting to access an array with an invalid index value (either negative or beyond the length of the array).	By the JVM
<code>ClassCastException</code> (Chapter 2)	Thrown when attempting to cast a reference variable to a type that fails the IS-A test.	By the JVM
<code>IllegalArgumentException</code>	Thrown when a method receives an argument formatted differently than the method expects.	Programmatically
<code>IllegalStateException</code>	Thrown when the state of the environment doesn't match the operation being attempted—for example, using a scanner that's been closed.	Programmatically
<code>NullPointerException</code> (Chapter 3)	Thrown when attempting to invoke a method on, or access a property from, a reference variable whose current value is <code>null</code> .	By the JVM
<code>NumberFormatException</code> (this chapter)	Thrown when a method that converts a <code>String</code> to a number receives a <code>String</code> that it cannot convert.	Programmatically
<code>ArithmeticException</code>	Thrown when an illegal math operation (such as dividing by zero) is attempted.	By the JVM
<code>ExceptionInInitializerError</code> (Chapter 2)	Thrown when attempting to initialize a static variable or an initialization block.	By the JVM
<code>StackOverflowError</code> (this chapter)	Typically thrown when a method recurses too deeply. (Each invocation is added to the stack.)	By the JVM
<code>NoClassDefFoundError</code>	Thrown when the JVM can't find a class it needs, because of a command-line error, a classpath issue, or a missing <code>.class</code> file.	By the JVM